*produces:*

```
[0, 3, 6, 9, 33, 66, 99, 111, 141, 171]
```

This also gives us a way to fix our definition of triangular numbers so that the user of that method doesn't have to know about the two-element array, we just use lazy and map to return only the number we care about:

```
triangular_numbers = Enumerator.produce([1, 2]) do |number, count|
  [number + count, count + 1]
end.lazy.map { _1.first }

p triangular_numbers.first(5)
```

*produces:*

```
[1, 3, 6, 10, 15]
```

## What's Next

Collections, blocks, and iterators are core concepts in Ruby. The more you write in Ruby, the more you'll find yourself moving away from conventional looping constructs. Instead, you'll write classes that support iteration over their contents. And you'll find that this code is compact, easy to read, and a joy to maintain. If this all seems too weird, don't worry. After a while, it'll start to come naturally. And you'll have plenty of time to practice as you use Ruby libraries and frameworks. Now, lets talk more about how Ruby lets you define and call methods.